CS341 Array Toolkit

How to use this sheet

This sheet lists predefined Java array operations and functions you are allowed to use. It does not tell you which problem needs which entry, and it does not tell you what strategy to build around them. That is part of the assignment: read each problem and work out for yourself which entries are useful and how to combine them.

- Not allowed in your main solution: Arrays.sort, any other sort function, and any structure based on hashing (HashMap, HashSet, and similar). Those are fine only in the optional stretch tier.
- Everything below works with plain arrays and 2D arrays only.
- Complexity assumes an array of length n, or a grid with r rows and c columns.

Entries

2D array creation and indexing

No import needed.

```
int[][] grid = new int[r][c];
int value = grid[i][j];
int rows = grid.length;
int cols = grid[0].length;
```

Access and assignment are $O(1)$. Allocating the grid itself is $O(rc)$.

array.length and grid.length / grid[0].length

No import needed. A field, not a method call, so it takes no parentheses.

```
int[] arr = {4, -2, 7, 5};
int n = arr.length;
```

$O(1)$.

Arrays.copyOfRange

```
import java.util.Arrays;

int[] arr = {4, -2, 7, 5, -1, 8, 3};
int[] slice = Arrays.copyOfRange(arr, 1, 4);
// slice now holds {-2, 7, 5}; end index is exclusive
```

$O(k)$ time and space, where k is the length of the slice.

Arrays.fill

```
import java.util.Arrays;

int[] arr = new int[10];
Arrays.fill(arr, -1);
```

$O(n)$ to fill an array of length n.

Integer.MAX_VALUE and Integer.MIN_VALUE

No import needed. Useful as a starting value before scanning for a minimum or maximum.

```
int best = Integer.MIN_VALUE;
for (int x : arr) {
    if (x > best) best = x;
}
```

$O(1)$ to read the constant; the surrounding scan is $O(n)$.

Math.abs

No import needed (part of java.lang, always available).

```
int distance = Math.abs(arr[i] - arr[j]);
```

$O(1)$.

Math.max and Math.min

No import needed.

```
int larger = Math.max(arr[i], arr[j]);
int smaller = Math.min(arr[i], arr[j]);
```

$O(1)$ per call.

System.arraycopy

No import needed (part of java.lang, always available).

```
int[] source = {1, 2, 3, 4, 5};
int[] destination = new int[5];
System.arraycopy(source, 0, destination, 0, source.length);
```

$O(k)$ to copy k elements.

Complexity reminders

- A loop nested inside another loop that both range over the full array or grid is $O(n^2)$ or $O((rc)^2)$, not $O(n)$, even if each individual step is $O(1)$.
- Calling Arrays.copyOfRange or System.arraycopy inside a loop that runs n times can turn an otherwise linear solution into a quadratic one.
- Allocating a new array or 2D array costs time and space proportional to its size, so creating one inside a loop adds up fast.